Beyond Sum-of-Sources

Six alternative formulations for fixed-weight ReLU expectations | 22 September 2026

No: sum-of-sources is not the only formulation. It is one representation of a particular cumulant recurrence. Other methods integrate the network function, represent its input-space dependence, optimize over admissible probability laws, or perform global approximate inference. None of the sources reviewed establishes your target: raw MSE $\approx 2\times 10^{-8}$ at $\leq 0.16B$.

I investigated six alternatives and ran two new diagnostics on one independently generated 1024×16 He-initialized network. The results distinguish **an alternative that exists mathematically** from **an affordable, accurate estimator**. The latter remains unresolved.

Alternative	Quantitative finding at n=1024
1. Activation-region / Gaussian polyhedral integration	One finite line restriction: 2,138 segments; about 62 u in float64.
2. Input-space Hermite / polynomial-chaos projection	Full degree-two state: 513.25 u per linear update alone.
3. Function-space tensor-train quadrature	Optimistic rank-2 scenario: about 132 u; small fit diagnostic failed.
4. Global moment / sum-of-squares optimization	One dense first-order moment-matrix factorization: about 819 u.
5. Certified expectation branch-and-bound	About 32 u per cell for two full backward affine bounds.
6. Fixed-weight EP / ML-VAMP inference	Standard dense-SVD setup: about 208 u; existing theory addresses a different target.

Three corrections to the quoted answer

Code-verified [1]. F51/F64 concern augmented K3 with K4=Sym(G⊗metric), a restricted harmonic channel. They do not establish that the unrestricted fourth cumulant can improve accuracy by only 9%. This correction does not make full K4 affordable.

Derived. A failed covariance, core or sampling implementation is not an impossibility theorem for its entire family. Likewise, 16M input weights and 1,100 labeled networks do not alone prove that every structured learned estimator is impossible. Your measured feature-fit failures still stand.

Derived. The leaders need not lie outside every familiar family. Cost coincidence and a 1.3× accuracy gap do not identify their algorithm. Also, 2% D21 accuracy is a requirement for the tested cumulant chain; a different global estimator only needs the final mean to be accurate.

Units: 1 u=2^{31} FLOPs; B=1024 u. The working target below is 160 u, slightly below 0.16B=163.84 u. Labels distinguish DERIVED, MEASURED HERE, CODE-VERIFIED and CONJECTURE. Public leaderboard and 10^9-sample truth figures are supplied by you; they were not reproduced.

1 / Integrate Gaussian activation regions

Derived; geometric tools exist [2,3]. Carry cones R_π and linear maps A_π, so $f_W(x)=A_\pi x$ on each activation region. Then

$$\mathbb{E} f_W(X) = \sum_\pi A_\pi \int_{R_\pi} x \phi_n(x)\, dx.$$

Update/state. At a layer, form WA_π, split each cone by its n sign inequalities, and zero inactive rows. Storage is $O(Rn^2)$ map entries plus facets. The dense map update alone costs $Rn^2(2n-1)/2^{31} \approx R$ u/layer; Gaussian cone moments are additional. Fixed signed weights enter every region and slope, so no odd information is ensemble-averaged away.

Cost obstruction. The first square Gaussian weight matrix is invertible almost surely, so all 2^{1024} first-layer sign cones are nonempty. This rejects explicit full enumeration, not implicit geometric integration. Holonomic Gaussian integration replaces integrals with differential equations, but its state depends on the polyhedron's face structure [3].

A cheaper geometric subproblem, actually tested

Restrict $x=y+tv$ with $y \perp v$. Store intervals and vector slopes/intercepts $f(y+tv)=a_r t+b_r$. Update the two coefficient matrices by W, then split at roots. An interval contributes exactly

$$a_r[\phi(l_r)-\phi(u_r)] + b_r[\Phi(u_r)-\Phi(l_r)].$$

FLOPs. Two n×n by n×R products cost ≈2R/n u/layer in float32, twice that in float64, plus splitting. State: 2nR floats. This is analytic integration across every gate on the chosen line, not a new name for point sampling.

Measured here: one line, t in [−6,6]	Result
Segments after layers 4 / 8 / 12 / 16	497 / 1,048 / 1,583 / 2,138
Float64 matrix products / arithmetic estimate	61.482 u / about 62.04 u
Final slope-and-intercept storage	33.41 MiB
Maximum error at 32 checked points	1.25×10^{-14}

First falsifier, executed. Count segments on one network before building a full integrator. This probe took 3.27 s; it leaves 1,023 Gaussian coordinates and the omitted tails unintegrated. It gives no final-mean accuracy claim. If remaining coordinates are sampled, the result is again a control-variate/Rao–Blackwell sampling method and must overcome your existing variance barrier. I do not recommend that variant.

2 / Project the network function in input space

Derived; polynomial-chaos machinery [4]. Carry $H_{i\alpha}$ for $h_i(x) \approx \sum_{\alpha \in \Lambda} H_{i\alpha} \Psi_\alpha(x)$, where Ψ are Gaussian-orthonormal Hermite polynomials. The mean is H_{i0}. Linear mixing is exact within the retained function space:

$$Z = WH, \qquad H^+_{i\alpha} = \mathbb{E}\left[\mathrm{ReLU}\left(\sum_\beta Z_{i\beta}\Psi_\beta(X)\right)\Psi_\alpha(X)\right].$$

State/cost. With P basis functions, state size is nP. The linear update costs $nP(2n-1)/2^{31} \approx P/n$ u/layer. Evaluating and projecting on S quadrature nodes adds approximately $2PS/n^2$ u/layer, plus basis construction. The nonlinear projection is not free.

Full degree two already has P=binom(1026,2)=525,825: **2.006 GiB** per coefficient state and **513.25 u per linear update**. Even tiling with the supplied Strassen rate is around 262 u. Thus the straightforward dense method fails before its nonlinear integration is priced.

What differs from the dead Hermite proposal. This represents the input-to-activation function, not diagonal quadratic coordinates of an old K3 query. Its errors come from omitted input interactions and numerical projection. The previous D21 spectra do not measure these coefficients.

First falsifier, under 0.01 u. For the cheap diagonal-quadratic variant, the retained fraction of a first-layer neuron's second-chaos energy is $\sum_j w_j^4 / (\sum_j w_j^2)^2$. On the same network it averaged **0.2926%**. This rejects a claim of faithful diagonal-quadratic function representation; it is not a lower bound on final-mean error. Adaptive bases remain a different, unpriced approximation until their projection step is specified.

3 / Tensorize the input integral, not K3

Derived; adaptive tensor quadrature exists [5]. On a product quadrature grid, represent each scalar output as

$$f_i(x_1, \ldots, x_n) \approx G_{i1}(x_1) \cdots G_{in}(x_n),$$

then integrate each core with its one-dimensional weights. Rank χ, q nodes and separate cores for all n outputs require $O(n^2 q \chi^2)$ entries: about 32 MiB at q=2, χ=2. Cross updates request selected full-network values and update cores. There is no layerwise cumulant update.

These tensor axes are input coordinates, not the three neuron indices of K3. Therefore the measured old-core TT rank≈250 is not a rank measurement of this object. The price is the network-evaluation budget, interpolation bias, and product-quadrature bias.

Function-space TT: a plausible bill is not enough

Conjectural cost gate. One fixed-rank pass using shared cross locations for all outputs requests roughly $N=nq\chi^2$ full forward evaluations. A forward pass costs exactly $L/n=1/64$ u including ReLUs, so the pass costs $Lq\chi^2$ u classically. Fully batching those requests permits the optimistic 0.51 multiplier on dense products.

At $q=2$, $\chi=2$: 128 u/pass classically, approximately **65.3 u/pass** with ideal batching. Two passes plus small-core arithmetic are roughly **132 u**. This is a conditional ledger, not the measured cost of an implemented solver: common pivots, stable tiny inverses and two-pass convergence are assumptions. Output-specific query sets or extra passes invalidate it. At $q=4$ or $\chi=4$ the analogous two-pass product bill is already about 261 u or 522 u.

Additional bias. Two-node Gauss–Hermite quadrature integrates Rademacher inputs, not Gaussian inputs. Multiplying by $E\|X\|/\sqrt{n}=0.999755889$ corrects the radius for a homogeneous network, but not the angular distribution. This unresolved bias must also be measured; a low TT rank would not settle it.

First falsifier: the unmeasured object can be probed cheaply

Measured here. Split the 1,024 input coordinates into two halves. Select 16 independent Rademacher settings for each half and evaluate all 256 combinations, obtaining a 16×16 matrix for every output. Compare its mean with the mean after its best rank-r Frobenius approximation.

Rank r	MSE of approximated grid mean	Function RMSE on grid
1	5.16×10^{-6}	0.2000
2	2.25×10^{-6}	0.1744
4	7.07×10^{-7}	0.1323
8	1.17×10^{-7}	0.0676

The complete diagnostic costs **<4.2 u**: 4 u for network evaluations and about 0.051 u for the 1,024 small SVDs, plus reconstructions. This is an exact finite-grid comparison, not MSE against the 10^9-sample Gaussian truth.

Interpretation. The budget-compatible rank-2 *Frobenius-fitting route* fails this small test by about 112× the target mean MSE. Mean-directed interpolation could behave differently; a rank-one constant can reproduce a mean if that mean is already known. Consequently this experiment is not an impossibility proof for all TT quadrature.

Measure-this, if developing that different fit. Before a full solver, test common-pivot rank-2 interpolation on four fresh 16×16 split grids, including Gaussian settings: <17 u, readily under one CPU hour. Require mean errors below 2×10^{-8} on the held-out finite-grid integrals; count all pivot-selection queries separately. Passing is necessary evidence for this prototype, not final Gaussian-accuracy validation.

4 / Optimize over the entire joint probability law

Derived application of moment/SOS methods [6]. Introduce all variables $z=(x,h^1,\dots,h^{16})$. ReLU is characterized by polynomial constraints $h\geq 0$, $h-Wh_{prev}\geq 0$ and $h\circ(h-Wh_{prev})=0$. Fix input moments to their Gaussian values and bound $E[h^{16}_i]$ using positive-semidefinite moment and localizing matrices.

State/update. This is a global optimization problem, not a forward chain. Increasing the relaxation order adds moment variables and constraints; an SDP solver updates them together. Signed W enters the constraints. This differs from truncating a forward cumulant recurrence, although both use moments.

FLOPs. The first moment matrix already has dimension $d=17n+1=17{,}409$ and occupies 1.129 GiB in float32. A single conventional dense Cholesky factorization costs $d^3/3\approx$ **819 u**; an SDP solve requires more. Sparse formulations change that ledger, but must separately establish tight bounds.

First falsifier, <0.01 u. For just $Y=\mathrm{ReLU}(Z)$, $Z\sim N(0,\sigma^2)$, the elementary degree-two moment relaxation permits $0\leq E[Y]\leq \sigma/2$, whereas the exact mean is $\sigma/\sqrt{(2\pi)}$. Thus its cheapest local version is far too loose already at layer one. This rejects that relaxation order, not the full hierarchy.

5 / Certified expectation bounds with branching

Derived adaptation of probabilistic verification [7]. Carry Gaussian input cells, their masses and first moments, and affine lower/upper network envelopes. Split a cell, propagate tighter ReLU relaxations, integrate the envelopes, then sum cell bounds. Alternatively integrate bounds on $P(f_i(X)>t)$ over t. Gaussian tails must be included.

State/cost. Two n-output affine envelopes need about $2n^2$ coefficients per active cell. Each backward layer uses two $n\times n$ products: about 2 u/layer/cell, or 32 u per full-depth cell; ideal Strassen reduces products toward 16.3 u. Branching, interval preparation, Gaussian masses and tails add cost. The working budget therefore buys only a few such full-output cells.

First falsifier, <12 u for leading operations. On one network, use eight cells and 32 selected outputs; backward products cost about 8 u, with roughly 0.5 u for interval passes. Bill all remaining work. For midpoint estimates, a certificate requires RMS bound width $\leq 2\sqrt{(2\times 10^{-8})}=$ **2.83×10^{-4}**. A wider gap rejects that eight-cell configuration. Source benchmarks do not demonstrate this tolerance at 1024×16.

6 / Fixed-weight expectation-consistent inference

Literature/code-verified [8]. EP and ML-VAMP operate on known weight matrices; they are not merely annealed formulas. Carry forward/backward Gaussian messages, update linear blocks by Gaussian conditioning and nonlinear blocks by truncated-Gaussian moment projection. A typical linear update has the form

$$\hat{z} = (\gamma I + \tau W^T W)^{-1}(\gamma r + \tau W^T s).$$

State/cost. Scalar-variance messages require O(Ln) entries plus weights and matrix decompositions. A four-matvec-per-layer sweep is about 4L/n=**0.0625 u**, excluding scalar updates. However, the inspected SVD-based linear primitive requires decompositions: a standard full-vector square SVD costs about 13 u under the inspected meter, or **208 u for 16 weights**. Matrix-free variants need a different complete ledger.

Why it is not a demonstrated solution. The cited theory concerns inference from observations and large-system reconstruction error, not a 10^{-4}-accurate prior mean on one fixed network. Without observations, the elementary Gaussian-message forward construction reduces to Gaussian moment propagation; bidirectional iteration alone does not create the missing information.

First falsifier, <1 u. Set the output likelihood to one and test the scalar ReLU site on actual-network Gaussian marginals. In the code's infinite-output-variance limit, the incoming Gaussian is unchanged and the outgoing mean is μΦ(μ/σ)+σφ(μ/σ). If the proposed whole algorithm adds no non-Gaussian joint state beyond these updates, your existing Gaussian-closure floor applies. Larger clusters or richer messages are new approximations that still need pricing.

What not to count as another working algorithm

Derived. Fourier transforms, heat equations, path integrals, Koopman operators and tropical geometry give useful equivalent representations. A representation becomes a computational proposal only after its discretization, compression and mean readout are specified. For example, ReLU networks admit differences of polytope support functions [9], but Minkowski sums and convex hulls can explode; evaluating their Gaussian widths is the unresolved integration task.

The parity obstruction concerns predicting an odd preactivation tensor from even statistics. It does not prohibit every fixed-W functional method or every even final-mean correction. Indeed E[ReLU(Z)]=(E[Z]+E|Z|)/2, and the second term is even under a final-row sign flip.

Research conclusion

Alternatives exist; no target-budget winner was found. Full geometric enumeration, dense polynomial chaos and dense moment optimization fail explicit cost gates. The cheap function-TT configuration has adverse new evidence. Verification and message passing require substantially tighter or richer approximations. The earlier table is not exhaustive, but neither this search nor those measurements justify naming a hidden leader algorithm.

Appendix A / Experimental scope and accounting

One network was generated with NumPy seed 22092026: 16 independent 1024×1024 float32 Gaussian He matrices. The two executed diagnostics use the complete network function, including every ReLU. They do not use the simplified MPP-only birth model criticized in the previous report.

Test	Measured / derived result	Scope
Input split-grid, all 1,024 outputs	256 evaluations = 4 u; complete probe <4.2 u	Best Frobenius rank truncation on a finite Rademacher grid.
Line restriction, all 16 layers	2,138 segments; ≈62.04 u; 3.27 s	Float64 slopes/intercepts on [−6,6]; not full Gaussian integration.
Line validation	32 points; max absolute discrepancy 1.25×10^{-14}	Additional direct forwards cost about 1 u in float64.
Diagonal second-chaos energy	Mean retained fraction 0.00292563	Computed from first-layer weight rows; no final-mean implication by itself.

The line arithmetic estimate includes both transported coefficient matrices, root construction and a conservative allowance for splitting and integration. Sorting comparisons and random instance generation are diagnostic overhead, not a shipping estimator bill. The 33.41-MiB figure covers final coefficient legs only; temporary copies and the weights use additional memory.

The complete diagnostic script ran in about 4.12 s here with one BLAS thread. Other CPUs will differ. None of these figures is a full V29-versus-10^9-sample-ground-truth comparison. No external repository was installed and benchmarked as a complete replacement estimator.

The Monte Carlo calibration in the quoted answer

Using your supplied adjusted MC plateau 1.2×10^{-6}, spending C=160 u implies raw MSE$\approx1.2\times10^{-6}/(160/1024)=7.68\times10^{-6}$. Matching 2×10^{-8} at that bill requires about **384×** variance reduction. At 0.1B the corresponding raw value is 1.2×10^{-5}, not 5×10^{-5}. The latter may belong to a different calibration; it should not be mixed with this plateau.

Why the scalar moment relaxation is loose

For σ=1, write m=E[Y] and s=E[Y²]=E[ZY]. PSD of the moment matrix for (1,Z,Y) requires $s-m^2-s^2\geq0$. Hence m≤1/2; m=0 and m=1/2 are feasible in this low-order relaxation. Positivity only at this relaxation order does not enforce the full ReLU graph almost surely. Higher orders are essential, and expensive.

Appendix B / Reproduce the input-function rank probe

This code reproduces the finite-grid comparison from scratch. It is deliberately a cheap numerical gate for a new representation, not a final estimator. It uses NumPy only; the geometric line test additionally used SciPy's standard normal CDF for interval integration.

```
import os
for key in ["OPENBLAS_NUM_THREADS", "OMP_NUM_THREADS",
            "MKL_NUM_THREADS"]:
    os.environ[key] = "1"
import numpy as np

n, L, m = 1024, 16, 16
rng = np.random.default_rng(22092026)
W = [rng.standard_normal((n,n), dtype=np.float32)
     * np.float32(np.sqrt(2/n)) for _ in range(L)]
U = rng.choice([-1., 1.], (n//2,m)).astype(np.float32)
V = rng.choice([-1., 1.], (n//2,m)).astype(np.float32)
X = np.concatenate([np.repeat(U,m,axis=1),
                    np.tile(V,(1,m))], axis=0)
for w in W:
    X = np.maximum(w @ X, 0)
Y = X.reshape(n,m,m)
A, s, Bt = np.linalg.svd(Y, full_matrices=False)
mu = Y.mean(axis=(1,2), dtype=np.float64)
for r in [1,2,4,8]:
    Yr = (A[:,:,:r] * s[:,None,:r]) @ Bt[:,:r,:]
    mur = Yr.mean(axis=(1,2), dtype=np.float64)
    print(r, np.mean((mur-mu)**2))
# 1: 5.162773e-6     2: 2.247895e-6
# 4: 7.067861e-7     8: 1.172560e-7
```

Reproduce the geometric update

Start with edges $(-6,6)$, slope $v/\|v\|$ and intercept y orthogonal to v. For each layer: multiply all slopes and intercepts by W; in each interval collect every root $-b_j/a_j$ lying strictly inside it; sort the roots; copy the parent coefficients into the new intervals; zero rows whose value at the interval midpoint is negative. Integrate the resulting coefficients using the formula on page 2.

Count matrix products using their actual interval counts R_{l-1}: $4\sum_l nR_{l-1}(2n-1)$ FLOPs in float64. The factor four is two products times the float64 surcharge. This protocol is independent of SyReNN's implementation and does not assume its wall-clock performance.

Primary papers and public implementations

The search used 27 targeted queries plus direct source inspection. The cited papers establish their respective methods; the FLOP estimates and benchmark-specific judgments in this report are derived here. None of these papers reports the requested 1024×16 leaderboard result.

[1] Existing-chain evidence: 504aldo, findings F51/F64; restricted harmonic K4 channel.

https://github.com/504aldo/whest-p2-cumulant-k3/blob/18c17e2d7a9aeacd399cfc2c6b571e4e16dbfb45/docs/findings_log.md

[2] Sotoudeh & Thakur (NeurIPS 2019), Computing Linear Restrictions of Neural Networks.

https://proceedings.neurips.cc/paper/2019/hash/908075ea2c025c335f4865f7db427062-Abstract.html

[2a] Public geometric-analysis implementation: SyReNN. Repository documentation reviewed; no end-to-end benchmark claimed.

https://github.com/95616ARG/SyReNN

[3] Koyama, Holonomic modules associated with multivariate normal probabilities of polyhedra.

https://arxiv.org/abs/1311.6905

[4] Xiu & Karniadakis (2002), The Wiener–Askey Polynomial Chaos for Stochastic Differential Equations.

https://doi.org/10.1137/S1064827501387826

[4a] Basis-adaptation code and scope: chaos_basispy. This is general-purpose machinery, not a validated neural-network estimator.

https://github.com/tsilifis/chaos_basispy

[5] Dolgov & Savostyanov (2019), Parallel cross interpolation for high-precision calculation of high-dimensional integrals.

https://arxiv.org/abs/1903.11554

[5a] Public tensor-cross quadrature implementation; demonstrated on Ising integrals, not this MLP benchmark.

https://github.com/savostyanov/ttcross

Primary papers and public implementations / continued

[6] Lasserre, Computing Gaussian & exponential measures of semi-algebraic sets.

https://arxiv.org/abs/1508.06132

[6a] Raghunathan, Steinhardt & Liang (2018), Semidefinite relaxations for certifying robustness to adversarial examples. The expectation formulation in this report is a derived adaptation.

https://arxiv.org/abs/1811.01057

[7] Boetius, Leue & Sutter (ICML 2025), Solving Probabilistic Verification Problems of Neural Networks using Branch and Bound.

https://proceedings.mlr.press/v267/boetius25a.html

[7a] Public probabilistic-verification code: probspecs. Output probability bounds, not a published 2e-8 mean estimator.

https://github.com/sen-uni-kn/probspecs

[8] Fletcher & Rangan (2017), Inference in Deep Networks in High Dimensions.

https://arxiv.org/abs/1706.06549

[8a] Inspected linear and ReLU estimation primitives in vampyre/estim.

https://github.com/GAMPTeam/vampyre/tree/master/vampyre/estim

[9] Zhang, Naitzat & Lim (ICML 2018), Tropical Geometry of Deep Neural Networks.

https://proceedings.mlr.press/v80/zhang18i.html

[10] Accounting reference: flopscope _flops.py, matmul_cost and svd_cost. Source inspected; all proposed-estimator totals still require metering.

https://github.com/AIcrowd/flopscope/blob/main/src/flopscope/_flops.py

How to read the evidence

The reference-chain correction comes from actual F51/F64 code documentation. VAMP’s linear/ReLU primitives were inspected. SyReNN, tensor-cross, basis-adaptation and probabilistic-verification repositories were reviewed for scope and algorithm descriptions. The executed experiments are independent NumPy diagnostics, not reproductions of their full software benchmarks.

A failed finite-grid test, an expensive implementation, or a loose relaxation closes that configuration. It does not prove that all methods in the corresponding mathematical family are impossible.